AutoBayes brms-parity loop: Tier E review

Status at the Tier E boundary, prepared for Andrew

19 October 2026

This is the Tier E boundary review. You asked for three things on 17 October: continue the loop, benchmark brms against AutoBayes, and rewrite the README. The benchmark (#68) showed AutoBayes 2.3 to 12.7 times faster end to end. The README now presents the library (#69), with your earlier text moved verbatim to `docs/agent-scaffold.md` and `docs/milestones.md`. All six Tier E features are on `main`, plus three follow-ups (#72, #79 and #78). Every merge was verified locally as `andrew-dou`; hosted CI is still refused for billing. Section 2 lists what brms gets wrong, and Section 6 is what needs you.

1 What was built

Table 1 lists the features.
AutoBayes now offers 39 of the 45 response families brms 2.23 documents; Tier E added ten. The six still missing are the multi-category families (`multinomial`, `dirichlet`, `dirichlet_multinomial`, `logistic_normal`), `cox` and `wiener`.

How close. I measured with the gates' own comparisons on `main` after the last feature merge (`4bac77b`). All 166 ungated cases pass, 33 of them from Tier E. Over the Tier E cases the worst relative errors are:

- 2.5×10^{-9} in G2, the gradients in a shape or precision computed through PyTensor's digamma;
- 1.0×10^{-15} in G3;
- 1.1×10^{-14} in predictions (`gen_extreme_value_varint`);
- 4.4×10^{-15} between the JAX and PyTensor densities;
- 2.9×10^{-13} in the post-processing tables (`weibull_censored_interval_post`).

Over all cases the worst G2 error is still Tier C's 7.2×10^{-9}. Everything is inside the unchanged tolerances. The full non-sampling suite on that `main` passed: 4,690 tests, no failure. `oracle/regen.py check -all` regenerated all 171 frozen fixtures from R and found every one identical.

2 What brms gets wrong

Tier E found more places where brms's own numbers are not the model's. Each is documented with numbers in its divergence row.

- **Truncated count means (D121).** With `trunc(lb = 1)`, brms's likelihood normalises by $P(Y \geq 1)$, but its `posterior_epred` returns $E[Y \mid Y \geq 2]$, summed over a finite support. At one frozen point brms gives 106.2, where $E[Y \mid Y \geq 1]$ is 134.0. E04 found this and checked it against the fixture's draws. AutoBayes computes the correct mean, and that feature is withheld pending your decision.

Feature	PR	What a user can now write	Diverg.
E06 links	#71, #79	`link=` on `ab.poisson` (`"sqrt"`), `ab.negbinomial` (`"identity"`), `ab.gamma` (`"inverse"`, `"identity"`), `ab.beta` (`"probit"`, `"cloglog"`), `ab.cumulative` (`"cloglog"`)	D132–D134
E05 prior draws, effects	#73	`ab.prior_draws(model)`; Savage–Dickey evidence ratios in `ab.hypothesis(..., prior=)`; `ab.conditional_effects(m, fit, ["x:z"])`; stacking built, withheld	D124–D127
E01 bounded, hurdle	#74	`ab.zero_one_inflated_beta`, `ab.zero_inflated_beta_binomial`, `ab.xbeta`, `ab.hurdle_cumulative`	D108–D111
E02 continuous	#75	`ab.inverse_gaussian`, `ab.von_mises`, `ab.asym_laplace` (quantile learned or fixed), `ab.frechet`, `ab.gen_extreme_value`	D112–D115
E03 contexts	#76	`["y ~ x", "shape ~ z"]` for the Weibull; formulas on `phi`, `zi`, `hu`, `alpha`; `weights()` for the Weibull and beta-binomial	D116–D118
E04 `cens()`, `trunc()`	#77, #78	`cens(c)` and interval `cens(c, y2)` for the Weibull, lognormal and Gamma families; `trunc(ub = b)` for counts	D120–D123

Table 1: The Tier E features. #70 froze their 33 brms cases; #72 fixed `log_lik` for the cloglog link.

- **Interval-censored Gamma gradient (D120).** brms's gradient in log(shape) is off by up to 2.8×10^{-5} relative. AutoBayes now accepts this case against a 50-digit reference of the model: `oracle/reference.py` gained the regularized incomplete gamma functions in decimal arithmetic.
- **Cumulative cloglog (D132).** brms forms the last category's probability as $1 - \text{inv_cloglog}(d)$ by subtraction, which loses every digit of an improbable category. Its log density is off by 5.9×10^{-7} at one frozen point. A log density may not fall back to a reference, so the link is accepted on a second, narrower case where brms is exact. The original case stays frozen and checked, with only its two log-density gates expected to fail; the test harness gained that per-gate option.
- **Stan's interval tails (E04's probes).** Stan's Weibull interval `(5, 6]` at shape 3 gives $-\infty$ where the mass is e^{-125}. Its Gamma(2, 1) interval `(40, 50]` gives -36.04 where the value is -36.29. AutoBayes takes every interval from the tail it lies in.
- **Smaller ones.**
  - Stan's von Mises density overflows above $\kappa \approx 714$ (D113).
  - brms's generalized extreme value mean is finite for $\xi \geq 1$, where the mean does not exist (D114).
  - loo's stacking weights stop 1.0×10^{-8} short of the optimum (D126); see the decisions.

3 What combining the features found

- **A silent `log_lik` error (D123).** Before E04, `ab.log_lik`, and therefore `loo` and `waic`, returned the uncensored, untruncated density even for the Gaussian family's `cens()` and `trunc()` models. It now applies both.

- **A `KeyError` on cloglog models.** `ab.log_lik` raised `KeyError: 'cloglog'` on every Bernoulli and binomial cloglog model. E06's agent spotted it by reading the code; #72 fixed it.
- **Shared parameter fields.** E01 and E02 both added a `kappa` prior field (xbeta's and the von Mises') and a parameter support on $(0, 1)$. They were merged into one of each, not duplicated.
- **Untested pairings, checked by hand.** E03 gave the Weibull a shape formula and `weights()`, and E04 gave it `cens()`. No frozen case covers either pair. Both match SciPy's censored Weibull row by row to 4×10^{-16}, and that check is now a test.
- **Two mistakes of mine, caught before merging.** A test helper of mine shadowed one of E04's. And my check scripts did not propagate pytest's exit status through `tail`, so a run with 7 failures reported success. I read the counts, fixed both, and every check script now sets `pipefail`.
- **A network outage.** It stopped all four wave-1 agents mid-work. I resumed each from its own context, and none lost work.

4 How the claims were checked, and their limits

- **Per feature.** The gate tool ran on the tree that merged, and the full suite ran on it or on the stack it merged with. The adapter and gate changes were read for weakening: all were extensions or stricter. Every earlier case model was unchanged.
- **Exceptions.** Two merges were verified by re-running only the test files concerned, not the whole suite: #72, a one-entry fix, and E04's two test-only fixes. The full suite on `main` afterwards passed.
- **Limits.** Nothing in the loop samples, so no Tier E model has been fitted beyond the benchmark's ten. `conditional_effects` is checked against brms only on its frozen cases; elsewhere it is checked for consistency with `predict()`.

5 Process and cost

Agent tokens reported:

- fixtures 0.41M;
- E02 0.82M, E01 0.78M, E03 0.73M, E04 0.65M, E06 0.59M, E05 0.57M;
- about 4.6M in all.

The rebases, conflict resolution, both follow-ups and both 50-digit references were mine. The suite lock kept one full suite on the machine at a time. That made the merge queue slower, at about 40 minutes per suite, but no run was distorted by another.

6 Decisions

1. **Stacking weights (D126).** Should two-reference acceptance cover them? brms's weights are 1.0×10^{-8} off the exact optimum, against the 10^{-9} prediction tolerance, because loo's optimiser stops there. Yes extends the P6 contract to `model_weights`; no keeps stacking withheld.
2. **Truncated count lower bounds (D121).** I recommend keeping the brms behaviour as a documented divergence and offering the correct mean. The alternative is accepting it against a reference, but this is a brms error in what it computes, not rounding.
3. **Reporting brms bugs.** The truncated count mean (D121) and the hurdle negative binomial draws (D86) look like brms bugs. They would go out under your account; I can draft the reports.
4. **Tier F.** Candidates, in the order I would take them:

(a) the multi-category families;
(b) the correlation structures `car`, `sar`, `fcor`, `unstr` and `cosy`;
(c) a first real fit of each family on the GPU or the cluster (needs your approval for compute);
(d) splines, once you decide the licensing.

5. **Still open from earlier tiers.**
   - Splines licensing.
   - CI billing.
   - The ArviZ `NOTICE`.